

Assignment 2 — FAST-NUCES Addendum

Performance Analysis on a Multi-Core CPU
Local adaptation of Stanford CS149 Assignment 1

Dr. Abdul Qadeer
Department of Computer Science, FAST-NUCES Lahore

Issued: 31 August 2026 • Due: 20 September 2026, 23:59 PKT

Read this before anything else

This document is *not* the assignment. The assignment is the Stanford CS149 Assignment 1 handout at <https://github.com/stanford-cs149/asst1>.

Read the Stanford handout first, cover to cover. Then read this addendum. Everything in this addendum **overrides** the Stanford handout wherever the two disagree — deadlines, machines, submission, targets, and the Program 6 dataset. Where this addendum is silent, the Stanford handout governs.

1 Logistics

Course	CS3006 — Parallel and Distributed Computing, Fall 2026
Assignment	Assignment 2 (Assignment 1 was the Linux kernel parallel-build exercise)
Issued	Monday, 31 August 2026
Due	Sunday, 20 September 2026, 23:59 PKT
Weight	100 points + 6 points extra credit
Collaboration	Individual. See Section 10.
Submission	Google Classroom only. See Section 9.
Late work	The course late-day policy stated in the course outline applies.
Questions	Google Classroom, or office hours (Thursday). TAs: Hashir Rauf (Section B), Minahil Azeem (Section A).

This is an individual assignment

The Stanford handout repeatedly mentions “groups”, “partners”, and adding a partner’s name to the submission. **Ignore all of it.** In CS3006 this assignment is done alone. You write your own code, run your own measurements on your own declared machine, and write your own analysis. Do not share code, measurement spreadsheets, plots, or draft write-ups with anyone.

Everything is assigned

All six programs are required: Program 1 through Program 6. There are no optional or dropped parts. The extra-credit items remain optional and are worth the points the Stanford handout states.

1.1 Suggested pacing

You have three weeks and three weekends. Almost all of the trouble students get into on this assignment comes from spending week one on setup and then discovering in the last two days that the measurement protocol takes real time. The schedule below is advice, not a requirement, but ignore it at your own risk.

By	You should have
Sun 6 Sep	A working Linux environment, ISPC 1.31.0 installed, the two build fixes applied, all six programs compiling, and <code>data.dat</code> generated with a matching checksum. If you are renting hardware, the account and payment method sorted by now.
Sun 13 Sep	Programs 1–5 written and correct, with a first pass of timings. Correctness first; do not start tuning until the output verifies.
Wed 16 Sep	Program 6 profiled, your f measured, your Amdahl ceiling computed, and your parallel version working.
Sat 19 Sep	Final measurement pass under the protocol in Section 5 — five runs per configuration, machine quiesced. Write-up drafted.
Sun 20 Sep	Submitted, with time to spare for a Google Classroom upload that fails the first time.

The single most common mistake

Leaving the measurements to the last day. Every timing in your write-up needs five repetitions on a quiet machine, and Program 6 alone takes tens of seconds per run. A full measurement pass across all six programs is an evening's work even when the code is finished and correct. Budget for it.

2 Platform: Linux only

You must do this assignment on a Linux system. This is a hard requirement, and it exists to keep the class from drowning in toolchain problems rather than learning about parallelism.

- **Supported:** a native Linux install (Ubuntu 22.04 or 24.04 recommended), a Linux virtual machine, a Linux cloud instance, or WSL2 with an Ubuntu image on Windows.
- **Not supported:** macOS, native Windows, Cygwin, MSYS2, Git Bash, online IDEs. If you use one of these, you are on your own — neither I nor the TAs will debug toolchain problems on unsupported platforms, and “it did not build on my Mac” is not an accepted reason for a late or incomplete submission.

WSL2 users: one extra caution

WSL2 works, but it virtualises the CPU topology and adds a filesystem layer. Keep the repository inside the Linux filesystem (`~/` or `/home/you/`), **not** under `/mnt/c/`, or your timings will be dominated by filesystem overhead. Also state clearly in your write-up that your measurements were taken under WSL2.

2.1 Prerequisites

```
sudo apt update
sudo apt install -y build-essential git wget python3 python3-pip python3-venv
```

2.2 Check that your CPU has AVX2

Programs 2, 3, 4 and 5 assume 8-wide AVX2 SIMD. Confirm your CPU supports it:

```
grep -o avx2 /proc/cpuinfo | head -1
```

If this prints `avx2`, you are fine. If it prints nothing, your CPU is either too old or is an ARM chip, and you must rent a machine (Section 8) or use an ARM configuration and declare it explicitly (Section 8.4).

2.3 Record your machine

Run these now and keep the output — you will paste it into your write-up:

```
lscpu | head -25
grep MemTotal /proc/meminfo
gcc --version | head -1
uname -a
```

3 Getting the code and the ISPC compiler

3.1 Starter code

```
git clone https://github.com/stanford-cs149/asst1.git
cd asst1
```

3.2 ISPC

The Stanford handout names ISPC v1.28.1. Use **v1.31.0** instead; it is the current release and I have verified that the entire assignment builds and runs with it.

```
wget https://github.com/ispc/ispc/releases/download/v1.31.0/ispc-v1.31.0-linux.tar.gz
tar -xzf ispc-v1.31.0-linux.tar.gz
export PATH=$PATH:$HOME/ispc-v1.31.0-linux/bin
ispc --version      # should report 1.31.0
```

Add the `export PATH` line to your `~/.bashrc` so it survives a new shell.

4 Required source fixes (do this before you build)

The Stanford starter code was written for an older compiler. On GCC 13 and later — which is what Ubuntu 24.04 ships — Program 1 will not compile, because newer `libstdc++` headers no longer pull in `<cstring>` and `<cstdlib>` transitively. Two one-line additions fix it. Make these edits before you do anything else:

File (under <code>prog1_mandelbrot_threads/</code>)	Add at the top	Error without it
<code>main.cpp</code>	<code>#include <cstring></code>	<code>'memset'</code> was not declared in this scope
<code>mandelbrotThread.cpp</code>	<code>#include <cstdlib></code>	<code>'exit'</code> was not declared in this scope

Programs 2–6 build cleanly as shipped. **Mention these two fixes in your write-up** — one sentence is enough. It is part of the record of what you actually ran.

Why this is worth noticing

This is not busywork. A five-year-old research artifact failing to compile against a current toolchain is the single most common obstacle to reproducing published performance results. You are about to spend two weeks producing performance numbers; noticing what it took to make somebody else's numbers reproducible is part of the lesson.

5 Hardware: you do not have a myth machine

The Stanford handout assumes their `myth` cluster: a 4-core, 8-thread Intel Core i7-7700K at 4.2–4.5 GHz with AVX2. Every numerical target in the handout — “7–8×”, “over 32×”, “2.1×” — is derived from that specific machine. You will be running on something else.

5.1 Machine Declaration (mandatory)

Your write-up must open with a **Machine Declaration** table. Missing or incomplete declarations cost **10 points off the total**, because without it your numbers cannot be interpreted at all.

Field	Example
CPU model	Intel Core i5-1135G7
Physical cores	4
Hardware threads (logical CPUs)	8
SMT / Hyper-Threading present?	Yes
Base / max clock	2.4 GHz / 4.2 GHz
Widest SIMD available	AVX2 (8-wide single precision)
RAM size and type	16 GB DDR4-3200, dual channel
Machine type	Own laptop / lab PC / cloud VM (name the provider and instance type)
OS, kernel, GCC version	Ubuntu 24.04, 6.8.0-40, GCC 13.3.0
ISPC version	1.31.0
Power state during measurement	Plugged in, performance governor

5.2 Measurement protocol (mandatory)

Laptops are much noisier measurement instruments than a lab cluster. Turbo boost, thermal throttling, and background processes can move a timing by a factor of two. Follow this protocol for every number you report, and state in your write-up that you did:

1. Close everything else — browsers, IDEs, Docker, virtual machines, Zoom. A single Chrome window can eat a core.
2. Plug the laptop in. Battery-saver modes cap clocks aggressively.
3. If available, set the CPU governor to performance:

```
sudo cpupower frequency-set -g performance
```
4. Run each configuration **at least 5 times** and report the **minimum** (this is what the Stanford timing harness already does within a run; do it across runs too). The minimum is the least noise-contaminated estimate of the machine's real capability.
5. Also report the spread — min and max — for at least one configuration per program, so the reader can judge how noisy your platform is.
6. Let the machine cool between long runs if you can hear the fan pinning. Thermal throttling on a thin laptop can silently halve your clock partway through a sweep.

Do not report a single run

A speedup computed from one serial run and one parallel run is not a measurement, it is an anecdote. Points are awarded for the analysis, and an analysis built on unrepeatable timings cannot be defended.

6 Revised performance targets

The Stanford targets stay in place *as targets for a myth machine*. For grading, each one is restated below relative to *your* declared hardware. In every case, the reasoning is what earns the points — a student who explains precisely why their machine falls short of a ceiling scores better than one who hits a number and cannot say why.

Throughout, let C = your physical core count, T = your hardware thread count, and W = your SIMD width in single-precision lanes (8 for AVX2).

Program	Stanford target	CS3006 target
Program 1, part 4	7–8 $\times$ on both views with 8 threads	Two things. (a) Show the per-thread timing evidence that your first decomposition is load-imbalanced and that your fixed decomposition is not. (b) Report speedup at T threads and compare it against the ideal T , explaining the gap. On a 4-core/8-thread machine you should approach the Stanford figure; on a 2-core/4-thread machine expect roughly 3–3.5 $\times$ and say so.

Program	Stanford target	CS3006 target
Program 1, part 5	Run with 16 threads	Run with $2T$ threads instead of a literal 16, and answer the same question.
Program 3, part 1	Maximum expected SIMD speedup	Your ceiling is W . State it, then explain why you observe less.
Program 3, part 2	Over $32\times$ with tasks	Your ceiling is $C \times W$ — for a 4-core AVX2 machine that is exactly the 32 the Stanford handout quotes. Report your achieved speedup as a fraction of $C \times W$ and account for the shortfall.
Program 4	Construct max/min speedup inputs	Unchanged. Report against your own W and C .
Program 5	Explain the performance	Unchanged, plus: look up your RAM's theoretical peak bandwidth (type, speed, channel count) and compare it to the GB/s the program reports. State both numbers.
Program 6	$\geq 2.1\times$	See below.

6.1 Program 6: a hardware-relative target

The Stanford $2.1\times$ figure assumes four cores. On a dual-core machine it is arithmetically unreachable by the intended solution, so the requirement is restated in terms of the law you already know:

1. Profile the code with `CycleTimer` and report the fraction f of total runtime spent in the hotspot you chose to parallelise.
2. Compute the Amdahl ceiling for your own machine,

$$S_{\max} = \frac{1}{(1-f) + \frac{f}{T}},$$

where T is your hardware thread count. Show the arithmetic.

3. Achieve at least $0.80 S_{\max}$. There is no absolute floor: if your machine's ceiling is low, then a modest speedup backed by the arithmetic showing it is near-optimal *is* the correct answer, and it scores full marks.
4. Say where your result sits relative to the Stanford $2.1\times$. On a 4-core/8-thread machine that figure is comfortably inside your ceiling; on two cores it is above it, and demonstrating that clearly is worth more than a number you cannot justify.

Worked example of the arithmetic

Suppose you profile the code and find that $f = 0.68$ of the runtime sits in the hotspot, and your machine has $T = 4$ hardware threads. Then

$$S_{\max} = \frac{1}{(1 - 0.68) + \frac{0.68}{4}} = \frac{1}{0.32 + 0.17} = \frac{1}{0.49} = 2.04,$$

so your requirement is $0.80 \times 2.04 = 1.63\times$. Notice that Stanford's $2.1\times$ is *above* this ceiling — on a 4-thread machine you cannot reach it by parallelising this hotspot alone, no matter how good your code is. Working that out and saying so is exactly the analysis being asked for. The numbers above are an illustration; use the f you measure, not this one.

Connection to the course notes

Step 2 is exactly the Amdahl calculation from the “Four Laws of Parallel Performance” notes, applied to a real profile you measured yourself instead of a textbook fraction. This is the point of the exercise: the serial fraction is not given to you, you have to go and find it, and everything downstream depends on getting it right. A good write-up here also comments on whether the Karp–Flatt metric computed from your measured speedups agrees with the f you measured directly — if the two disagree, that disagreement is itself a finding worth a paragraph.

7 Program 6: generating data.dat yourself

The Stanford handout tells you to symlink an 800 MB `data.dat` out of Stanford's AFS filesystem. You have no access to it, and distributing an 800 MB file through Google Classroom is not practical. Instead you will generate the dataset locally with the script `generate_data.py`, which I am distributing with this addendum on Google Classroom.

The generator is **deterministic**: it uses a fixed seed, so the same command produces a byte-identical file on every machine and on every run. Everyone in the class is therefore timing the same workload, and your results are comparable to your classmates' and to mine.

7.1 How to run it

```
cd asst1/prog6_kmeans
python3 -m venv venv
source venv/bin/activate
pip install numpy

# Full-scale dataset. This is the one your reported numbers must use.
python3 generate_data.py

ls -l data.dat          # expect 804,002,420 bytes (766.8 MiB)
md5sum data.dat         # expect 3a25f24193f4fdca82ee4cb2737fd5bb
```

Report the checksum

Paste the `md5sum` output into your write-up. If it does not match `3a25f24193f4fdca82ee4cb2737fd5bb`, your dataset differs from everyone else's, your timings are not comparable, and you should fix that before going further. Post on Google Classroom if you cannot get a match.

7.2 Resource requirements

	Needed	Measured on my reference machine
Free disk space	≥ 1 GB	766.8 MiB output file
Peak RAM during generation	≥ 2 GB free	1.65 GB resident
Generation time	—	10.5 s on a single 2.1 GHz core

The k-means program itself allocates roughly another 800 MB when it reads the file, so you want at least 4 GB of free RAM to run Program 6 comfortably. If your machine swaps, your timings become meaningless — you will be measuring your SSD, not your CPU.

7.3 A smaller dataset for debugging only

The full workload takes tens of seconds per run on a single core, and you will run it many times. While you are getting your code *correct*, work with a smaller file:

```
python3 generate_data.py --M 100000 --output data_small.dat
cp data_small.dat data.dat      # main.cpp always reads ./data.dat
```

Switch back before you measure

Every timing and speedup in your write-up must come from the full $M = 1,000,000$ dataset. Numbers taken at reduced M are not comparable and will not be accepted. Re-generate the full file and re-verify the checksum before your final measurement pass.

7.4 Plotting: fix the pinned requirements

The file `prog6_kmeans/requirements.txt` pins `matplotlib~3.6.0`, which has no wheel for Python 3.12 and fails to build from source on Ubuntu 24.04. Ignore the pin and install current versions:

```
pip install "numpy>=1.19.5" "matplotlib>=3.8" "scikit-learn>=1.1.2"
python3 plot.py
```

This produces `start.png` and `end.png`. Include both in your write-up as evidence that your optimised code still clusters correctly. The Stanford handout's warning applies to our generated data too: the 100-dimensional points are projected to two dimensions by PCA, so a point can look like it is assigned to the “wrong” centroid in the picture while being correctly assigned in the real space. What you are checking is that `end.png` shows three coherent clusters with centroids sitting in them.

8 Renting a machine

You are not required to rent anything. If your own Linux machine has AVX2, use it — declare it and report your numbers. But some of you will want hardware closer to what the Stanford

handout assumes, and some of you may not have a suitable machine at all. This section tells you what that costs.

What “closer to myth” actually means

The topology that matters for this assignment is **4 physical cores with 2 hardware threads each** (8 logical CPUs), with AVX2. Program 1 specifically asks you to observe hyper-threading effects, so an instance that exposes SMT sibling threads reproduces the intended experiment most faithfully. On the major clouds, an “8 vCPU” x86 instance is normally 4 physical cores $\times$ 2 threads — exactly the shape you want.

8.1 Cost estimates

A well-organised student needs roughly **8–10 hours** of instance time for this whole assignment: build once, then run the sweeps, then re-run after each code change. Budget 20 hours if you expect to iterate a lot. The table below gives per-hour costs and a 10-hour total.

Option	Shape	PKR/hr	10 hrs	Fidelity
Your own Linux machine	whatever you have	0	0	Declare it
FAST lab machine	lab spec	0	0	Declare it
Google Cloud, \$300 free credit (90 days), 8-vCPU x86	4c / 8t, AVX2	0	0	Excellent
DigitalOcean, \$200 credit via GitHub Student Pack	8 dedicated vCPU	0	0	Very good
Hetzner CX43	8 shared vCPU	8	$\approx$ 82	Poor (noisy)
Hetzner CCX33	8 dedicated vCPU	72	$\approx$ 715	Very good
AWS <code>c7i.2xlarge</code> , spot	4c / 8t, AVX-512	30	$\approx$ 300	Excellent
AWS <code>c7i.2xlarge</code> , on-demand	4c / 8t, AVX-512	99	$\approx$ 992	Excellent

List prices as published on 30 August 2026: AWS `c7i.2xlarge` on-demand \$0.357/hr (us-east-1); spot assumed at 70% below on-demand, which is typical but not guaranteed. Hetzner CX43 €0.0256/hr, CCX33 €0.2219/hr (Germany/Finland, excluding VAT). Converted at USD 1 = PKR 278 and EUR 1 = PKR 322, mid-market rates on 30 August 2026. **All of these numbers move.** Check the provider’s own pricing page before you commit, and re-check the exchange rate.

8.2 The cheapest sensible route

1. **Try your own machine first.** If `grep avx2 /proc/cpuinfo` succeeds and you have 4 GB of free RAM, you are done. This costs nothing and is a perfectly valid submission.

2. **GitHub Student Developer Pack.** Free with your FAST email address, and it bundles DigitalOcean credit (\$200 at the time of writing) plus other cloud credits. Sign up at <https://education.github.com/pack>. This is the best value available to you and it is worth doing regardless of this assignment.
3. **Google Cloud free trial.** \$300 in credit valid for 90 days for new accounts. That is far more than this assignment needs. A card is required for identity verification, but you are not charged unless you manually upgrade to a paid account.
4. **Hetzner CX43** if you want to pay cash and pay as little as possible — under PKR 100 for the whole assignment. The catch is that CX instances share physical cores with other tenants, so your timings will be noisy and your speedup curves will look ragged. If you go this route, run more repetitions and say clearly in your write-up that you were on shared vCPUs.

Four ways to lose money

1. **Forgetting to delete the instance.** On Hetzner, a *stopped* server still bills — you must *delete* it. On AWS, *stopped* instances stop billing compute but keep billing for their EBS volume. Delete everything the moment you are finished, and check the billing page the next day.
2. **Set a budget alert** on day one. Every provider has them. A forgotten instance running for a month is the classic way a PKR 100 assignment becomes a PKR 10,000 problem.
3. **Spot instances can be reclaimed** with two minutes' notice. That is fine for this workload — nothing here runs for hours — but save your code to Git, not just to the instance's disk.
4. **International payment.** You need a card enabled for international transactions. Sort this out early if you are planning to pay for compute; discovering on 19 September that your card is blocked is not a reason for an extension.

8.3 What to avoid

- **Burstable instances** — AWS `t3/t4g`, GCP `e2-micro`, and anything described as “shared” or “burstable” — throttle your CPU once a credit balance runs out. Your speedup curve will measure the provider's throttling policy rather than your program.
- **Oracle Cloud's Always Free tier.** It is genuinely free and genuinely useful, but the always-free compute is Ampere ARM. No AVX2. See Section 8.4.
- **Free-tier micro instances** with one vCPU. Program 1 is about multi-core scaling; you cannot do it on one core.

8.4 If you end up on ARM

ARM machines (Apple Silicon under a Linux VM, AWS Graviton, GCP Axion, Oracle Ampere, Raspberry Pi) do not have AVX2. They have NEON, typically 4-wide for single precision. The assignment still works, but:

- Read `README_aarch64.md` in the Stanford repository first.
- Build ISPC targets with `-target=neon-i32x4` (or `neon-i32x8` where supported) instead of `avx2-i32x8`.

- Your SIMD width W changes, so every ceiling in the target table changes with it. State W explicitly and derive your ceilings from it.
- Say “I ran on ARM” in the very first line of your Machine Declaration.

An ARM submission is acceptable and will be graded on the same analytical standard. An ARM submission that quietly compares itself against AVX2 ceilings will not.

9 What to submit

You will submit **your whole Git repository**, cleaned of build output and zipped. Not a handful of renamed `.cpp` files — the entire working tree plus its history. There is no Gradescope in this course; ignore every Gradescope instruction in the Stanford handout.

Why a repository

Two reasons. First, all six programs have code worth reading — the input vectors you construct for Program 4 and your ISPC changes in Program 5 are part of your answer, and the Stanford submission list omits them. Second, your commit history shows how you worked: what you tried, what you abandoned, and when. A history of small commits across several days is the clearest possible evidence that the work is yours, and it is exactly what a copied solution cannot produce. Treat this as practice for how performance work is done and reviewed in industry.

9.1 Setting up your repository (do this on day one)

Start your own history from the pristine starter code, so that every later commit is visibly your work:

```
git clone https://github.com/stanford-cs149/asst1.git cs3006-a2
cd cs3006-a2
rm -rf .git # discard Stanford's history
git init
git add -A
git commit -m "Initial commit: unmodified CS149 starter code"
```

Now create a `.gitignore` at the repository root, **before you build anything**:

```
cat > .gitignore <<'EOF'
objs/
*.o
*.ppm
*.log
*.dat
venv/
__pycache__/_
mandelbrot
mandelbrot_ispc
sqrt
saxpy
kmeans
myexp
ispc-*/
ispc-*.tar.gz
EOF
git add .gitignore && git commit -m "Add gitignore for build output and dataset"
```

Never commit data.dat

It is 767 MiB. If you commit it once, it lives in your Git history forever — deleting the file later does *not* shrink the repository, and your submission will be unuploadable. This is why the `.gitignore` goes in before your first build.

If you have already committed it, do not try to surgically excise it. Copy your source files somewhere safe, delete `.git`, and start a fresh repository from the starter code with the `.gitignore` in place, re-committing your work in a few honest steps. Say in your write-up that you had to do this.

9.2 How to commit

Commit as you work, not once at the end. Concretely, I expect to see at least **ten commits spread across at least four different days**, with messages that say what changed and why — "Row-cyclic decomposition fixes thread 3 imbalance", not "update". A repository whose entire content arrives in one commit on 19 September will be treated as a red flag and may be referred under Section 10.

Failed experiments are welcome. If you tried parallelising `computeCost` and it did not help, commit that, then commit the revert. That is a better submission than a clean history that hides the search.

9.3 Cleaning up before you zip

Build output is roughly 34 MB of the repository — object files, two 5.5 MB PPM images from Program 1, two more from Program 3, and two 8.5 MB k-means log files. Strip all of it.

Save your plots first

`make clean` in `prog6_kmeans` deletes `*.png`, which includes your `start.png` and `end.png`. Copy them out *before* cleaning.

```
cd cs3006-a2

# 1. Rescue the k-means plots
mkdir -p plots
cp prog6_kmeans/start.png prog6_kmeans/end.png plots/

# 2. Clean every program
for d in prog1_mandelbrot_threads prog2_vecintrin prog3_mandelbrot_ispc \
    prog4_sqrt prog5_saxpy prog6_kmeans; do (cd $d && make clean); done

# 3. Remove the dataset, stray output, and the Python venv
rm -f prog*/*.ppm prog*/*.log prog*/*.dat
rm -rf prog6_kmeans/venv prog*/__pycache__

# 4. Commit the final state and tag it
git add -A
git commit -m "Final submission"
git tag -a submission -m "CS3006 Assignment 2 final submission"

# 5. Write a plain-text history for the TAs
git log --oneline --stat > gitlog.txt

# 6. Check the size, then zip
du -sh . # expect roughly 3 MB
```

```
cd ..
zip -r CS3006_A2_23L-1234.zip cs3006-a2
ls -lh CS3006_A2_23L-1234.zip      # expect roughly 2-3 MB
```

Size targets

A properly cleaned repository is about **3 MB** on disk and zips to roughly **2–3 MB** — I measured 3.2 MB and 2.4 MB on a full run through the sequence above. Anything under **5 MB** is healthy. If your archive is over **10 MB** you have left something in; check for `.ppm` files, `.log` files, `objs/`, or a `venv/`. If it is over **50 MB** you have almost certainly committed `data.dat` or a build directory into Git history, and cleaning the working tree will not fix it — see the warning above.

For reference, an uncleaned repository after a full build and a run of every program is about **20 MB**, most of it four PPM images and two k-means log files.

9.4 What the archive must contain

Name it `CS3006_A2_<rollnumber>.zip`, for example `CS3006_A2_23L-1234.zip`.

<code>.git/</code>	Your full commit history. Do not delete it — without it the submission is incomplete.
<code>writeup.pdf</code>	At the repository root. PDF only — not <code>.docx</code> , not <code>.odt</code> . Your name and roll number on page 1.
<code>gitlog.txt</code>	Output of <code>git log --oneline --stat</code> .
<code>plots/</code>	<code>start.png</code> and <code>end.png</code> from Program 6.
<code>prog1_...</code> through <code>prog6_...</code>	All six program directories with your sources, cleaned of build output.
<code>extra/</code>	Any extra-credit code, if you attempted it. Point to it from the write-up.

Verify before you upload

Unzip your archive into a fresh directory and run:

```
cd cs3006-a2 && git log --oneline | head && git tag && ls -d prog*/
```

You should see your commits, the `submission` tag, and all six program directories. A submission that unzips into an empty or broken repository cannot be graded, and “it worked on my machine” does not apply to an archive.

9.5 Write-up structure

Follow this order. It makes grading consistent and it makes your own argument easier to follow.

1. **Machine Declaration** (Section 5) and a statement that you followed the measurement protocol.
2. **Build notes:** the two include fixes, your ISPC version, and anything else you had to change to get the code to compile.

3. **Program 1:** speedup graph for view 1 (and view 2 if you produced it), per-thread timing evidence, your decomposition, the $2T$ -thread result.
4. **Program 2:** vector utilisation across widths 2, 4, 8, 16, and your explanation of the trend.
5. **Program 3:** both parts, with speedups expressed against W and $C \times W$.
6. **Program 4:** your max-speedup and min-speedup inputs, with reasoning.
7. **Program 5:** measured GB/s versus your RAM’s theoretical peak.
8. **Program 6:** the `md5sum` of your `data.dat`, your profiling narrative, your f , your Amdahl ceiling, your achieved speedup, and `start.png` / `end.png`.
9. **Extra credit**, if attempted, clearly labelled.

How to write the Program 6 narrative

The Stanford handout asks for a sequence of steps: “I measured ... which led me to believe X. So I tried ... resulting in a speedup of ...” Take that seriously. A write-up that reports only the final answer earns far fewer points than one that shows a wrong hypothesis being tested and discarded. Measurement that changed your mind is the most valuable thing you can put in this document.

9.6 Grading

Points follow the Stanford handout: Program 1 (20), Program 2 (20), Program 3 (20), Program 4 (15), Program 5 (10), Program 6 (15), for 100 points, plus up to 6 points of extra credit. Within each program, the majority of the marks are for *analysis*, not for hitting a number.

Deductions specific to CS3006:

Missing or incomplete Machine Declaration	–10
Missing <code>.git/</code> history, or a single all-at-once commit	–10
No statement of measurement protocol, or single-run timings only	–5
Program 6 numbers taken at reduced M , or missing <code>md5sum</code>	–5
Write-up submitted in a format other than PDF	–5
Build output or <code>data.dat</code> left in the archive	–5

10 Academic integrity

This is an individual assignment and the university’s academic integrity policy applies in full.

- Solutions to CS149 Assignment 1 are all over GitHub. Copying them is plagiarism, it is easy to detect, and it defeats the entire purpose — the measurements are the assignment, and you cannot copy a measurement taken on somebody else’s hardware and truthfully declare it as your own.
- Do not share code, plots, spreadsheets, or draft write-ups with classmates. Discussing concepts is fine; exchanging artifacts is not.
- If you use an AI assistant, you must say so in your write-up: state which tool and what you used it for. Using one to help you understand ISPC’s execution model is fine. Using one to generate your measurements, your analysis, or your conclusions is not, and fabricated numbers are treated as academic misconduct.

- Every number in your write-up must be one you actually measured on the machine you declared.

11 Quick start

```
# 1. Prerequisites
sudo apt update && sudo apt install -y build-essential git wget python3 \
python3-pip python3-venv
grep -o avx2 /proc/cpuinfo | head -1          # must print: avx2
lscpu | head -25 > machine.txt                # keep this for your write-up

# 2. ISPC
wget https://github.com/ispc/ispc/releases/download/v1.31.0/ispc-v1.31.0-linux.tar
.gz
tar -xzf ispc-v1.31.0-linux.tar.gz
echo 'export PATH=$PATH:$HOME/ispc-v1.31.0-linux/bin' >> ~/.bashrc
source ~/.bashrc && ispc --version

# 3. Starter code, as your own repository (see Section 9.1)
git clone https://github.com/stanford-cs149/asst1.git cs3006-a2 && cd cs3006-a2
rm -rf .git && git init && git add -A
git commit -m "Initial commit: unmodified CS149 starter code"
# now add .gitignore per Section 9.1, BEFORE building anything

# 4. Apply the two required include fixes (Section 4), then build
sed -i '1i #include <cstring>'  prog1_mandelbrot_threads/main.cpp
sed -i '1i #include <stdlib.h>' prog1_mandelbrot_threads/mandelbrotThread.cpp
for d in prog1_mandelbrot_threads prog2_vecintrin prog3_mandelbrot_ispc \
    prog4_sqrt prog5_saxpy; do (cd $d && make); done

# 5. Program 6 dataset
cd prog6_kmeans
python3 -m venv venv && source venv/bin/activate
pip install numpy "matplotlib>=3.8" "scikit-learn>=1.1.2"
python3 generate_data.py
md5sum data.dat          # must be 3a25f24193f4fdca82ee4cb2737fd5bb
make && ./kmeans
```

Attribution and distribution

The underlying assignment — the starter code, the six programs, and the questions — is the work of the Stanford CS149 course staff and is used here for teaching purposes with attribution. See <https://github.com/stanford-cs149/asst1>. No Stanford text or figures are reproduced in this document; this addendum consists solely of local instructions written for CS3006.

This addendum, the `generate_data.py` generator, and all associated CS3006 course materials are prepared by **Dr. Abdul Qadeer** for students enrolled in CS3006 at FAST-NUCES Lahore, Fall 2026. **Do not copy. Do not distribute.** These materials are not to be reposted, uploaded to file-sharing or tutoring sites, or shared outside the enrolled class in any form.